

Summary and Reflections Report

1. Summary

- a. Describe your unit testing approach for each of the three features.
- i. To what extent was your approach aligned to the software requirements? Support your claims with specific evidence.

Module 3 Software Requirements

Contact Class Requirements

- The contact object shall have a required unique contact ID string that cannot be longer than 10 characters. The contact ID shall not be null and shall not be updatable.
- The contact object shall have a required firstName String field that cannot be longer than 10 characters. The firstName field shall not be null.
- The contact object shall have a required lastName String field that cannot be longer than 10 characters. The lastName field shall not be null.
- The contact object shall have a required phone String field that must be exactly 10 digits. The phone field shall not be null.
- The contact object shall have a required address field that must be no longer than 30 characters. The address field shall not be null.

Contact Service Requirements

- The contact service shall be able to add contacts with a unique ID.
- The contact service shall be able to delete contacts per contact ID.
- The contact service shall be able to update contact fields per contact ID. The following fields are updatable:
 - firstName
 - lastName
 - Number
 - Address

Module 4 Software Requirements

Task Class Requirements

- The task object shall have a required unique task ID String that cannot be longer than 10 characters. The task ID shall not be null and shall not be updatable.
- The task object shall have a required name String field that cannot be longer than 20 characters. The name field shall not be null.
- The task object shall have a required description String field that cannot be longer than 50 characters. The description field shall not be null.

Task Service Requirements

- The task service shall be able to add tasks with a unique ID.
- The task service shall be able to delete tasks per task ID.
- The task service shall be able to update task fields per task ID. The following fields are updatable:
 - Name
 - Description

Module 5 Software Requirements

Appointment Class Requirements

- The appointment object shall have a required unique appointment ID string that cannot be longer than 10 characters. The appointment ID shall not be null and shall not be updatable.
- The appointment object shall have a required appointment Date field. The appointment Date field cannot be in the past. The appointment Date field shall not be null.
Note: Use java.util.Date for the appointmentDate field and use before(new Date()) to check if the date is in the past.
- The appointment object shall have a required description String field that cannot be longer than 50 characters. The description field shall not be null.

Appointment Service Requirements

- The appointment service shall be able to add appointments with a unique appointment ID.
- The appointment service shall be able to delete appointments per appointment ID.

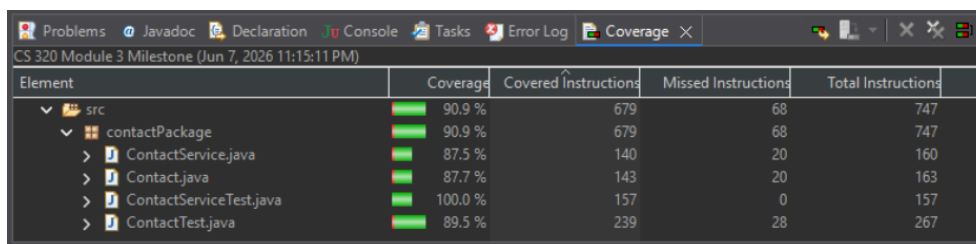
My testing approach was highly aligned with the software requirements for each case. I tested each attribute in the Contact, Task, and Appointment classes to ensure that the

code behaved properly for entries that were within bounds and out of bounds.

Furthermore, I tested the “getters and setters” in each class to verify that out of bound entries could not be set and that get operation functioned correctly. As it pertains to the service classes, the addition, deletion, and update (Module 3 & 4 only) functions were tested to verify functionality as well.

- ii. Defend the overall quality of your JUnit tests. In other words, how do you know your JUnit tests were effective based on the coverage percentage?

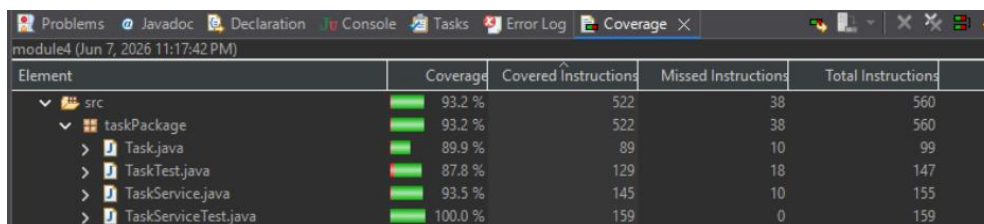
Module 3 JUnit test coverage percentages



CS 320 Module 3 Milestone (Jun 7, 2026 11:15:11 PM)

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
src	90.9 %	679	68	747
contactPackage	90.9 %	679	68	747
ContactService.java	87.5 %	140	20	160
Contact.java	87.7 %	143	20	163
ContactServiceTest.java	100.0 %	157	0	157
ContactTest.java	89.5 %	239	28	267

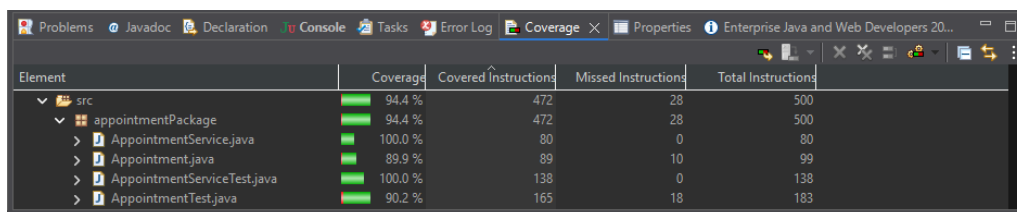
Module 4 JUnit test coverage percentages



module4 (Jun 7, 2026 11:17:42 PM)

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
src	93.2 %	522	38	560
taskPackage	93.2 %	522	38	560
Task.java	89.9 %	89	10	99
TaskTest.java	87.8 %	129	18	147
TaskService.java	93.5 %	145	10	155
TaskServiceTest.java	100.0 %	159	0	159

Module 5 JUnit test coverage percentages



Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
src	94.4 %	472	28	500
appointmentPackage	94.4 %	472	28	500
AppointmentService.java	100.0 %	80	0	80
Appointment.java	89.9 %	89	10	99
AppointmentServiceTest.java	100.0 %	138	0	138
AppointmentTest.java	90.2 %	165	18	183

Overall, I believe the quality of the JUnit tests was high based on the coverage percentage of each test. The percentages for each test were all above 87% with no errors

in testing for any of the classes, so the tests were constructed correctly and covered a wide variety of inputs.

- b. Describe your experience writing the JUnit tests.
- i. How did you ensure that your code was technically sound? Cite specific lines of code from your tests to illustrate.

Amongst the various test cases, I believe the code in lines 10-70 of the “ContactServiceTest” class, lines 9-80 of the “TaskServiceTest” class, and lines 10-63 of the “AppointmentServiceTest” confirm that the code is technically sound and performs the required operations without error.

- ii. How did you ensure that your code was efficient? Cite specific lines of code from your tests to illustrate.

To verify efficiency, I performed a timeout assertions test using an addition function in lines 56-61 of the “ContactServiceTest” class, performed the same test in lines 45-51 of the “TaskServiceTest” class, and conducted the timeout assertion test in lines 46-52 of the “AppointmentServiceTest” class as well.

2. Reflection

- a. Testing Techniques-
 - i. What were the software testing techniques that you employed in this project? Describe their characteristics using specific details.

I implemented JUnit testing on the three applications, JUnit testing is a form of dynamic testing focuses on the software that's being developed and encompasses a myriad of testing methodologies such as requirements-based or black-box testing, white-box testing, and experience-based testing. These methods center around determining test bases, test conditions, and test cases to incrementally inspect things like functional requirements, components, integrated systems, and the logic that the executable code is comprised of. Dynamic testing is essentially used to ensure that a system's design aligns with its specifications, and that the system as a whole works as intended.

- ii. What are the other software testing techniques that you did not use for this project?

Describe their characteristics using specific details.

Unlike the dynamic testing techniques, static testing wasn't conducted for the project. Static testing involves the examination of work products such as user stories, contracts, requirements or specification documents, or code without the actual execution of said code. Because this sort of testing doesn't require code execution, it can be implemented earlier in the development life cycle, potentially fixing issues before they have more costly ramifications later in the development process. Approaches to static testing typically revolve around informal and formal reviews, wherein the work products under review are either given a cursory inspection or a more rigorous, detailed inspection with documented results.

- iii. For each of the techniques you discussed, explain the practical uses and implications for different software development projects and situations.

Both techniques that were discussed are used within the software development life cycle, but dynamic testing is useful for identifying runtime issues such as memory leaks, incorrect logic, performance bottlenecks, and unexpected interactions between components. This type of testing is particularly important for web applications, mobile applications, and systems that interact with users or external services because it verifies that the software behaves as expected. As Hambling et al. (2019) explains, static testing is useful for catching defects in work products such as non-functional requirements, user stories, and user guides before they are deployed in code avoiding costly fixes later in the SDLC.

b. Mindset

- i. Assess the mindset that you adopted working on this project. In acting as a software tester, to what extent did you employ caution? Why was it important to appreciate the complexity and interrelationships of the code you were testing? Provide specific examples to illustrate your claims.

Throughout the entire project, I wrote 61 test cases amongst the three applications and reviewed each JUnit test class multiple times to ensure that the test cases were executed properly. Exercising that level of caution when examining code that I've written is something I haven't done previously; it made me consider how consequential the role of a software tester can be when operating on a codebase that's nearing deployment, let alone software that'll be implemented in an environment or business that's subject to strict regulations. Complex code and the interrelationships inherent in said code require extreme care and heightened awareness where testing is concerned. Some of the most

catastrophic software defects in recent history, such as Japan's Hitomi satellite (Academy, 2016) or the European Space Agency's Ariane 5 (The Explosion of the Ariane 5, 2021) rocket might have been avoided with more comprehensive testing practices that thoroughly accounted for things like satellite thruster configuration or numerical data conversions, respectively.

- ii. Assess the ways you tried to limit bias in your review of the code. On the software developer side, can you imagine that bias would be a concern if you were responsible for testing your own code? Provide specific examples to illustrate your claims.

To limit bias in my review of the code, I utilized the same JUnit tests in each application. Furthermore, I attempted to maintain consistent test coverage percentages amongst the three applications to ensure that each application received sufficient testing. In my opinion, bias can certainly arise when testing code that you've written yourself due to the attachments that we as people form with our own work in combination with overestimations in our own perceived level of skill/knowledge.

- iii. Finally, evaluate the importance of being disciplined in your commitment to quality as a software engineering professional. Why is it important not to cut corners when it comes to writing or testing code? How do you plan to avoid technical debt as a practitioner in the field? Provide specific examples to illustrate your claims.

Along with the buildup of technical debt, cutting corners almost certainly increases the possibility of introducing a fatal defect(s) into the production version of an application, whether writing or testing the code. As a future practitioner in the field of computer

science, I aim to be diligent when it comes to working on software, receptive to the advice of my peers as well as the guidance from senior colleagues, and proactive in my approach to problem solving. I believe these qualities, together with the software testing principles covered throughout this course, will bolster my ability to prioritize code quality throughout the development process.

References:

Hambling, B., Morgan, P., Samaroo, A., Thompson, G., & Williams, P. (2019). *Software testing : An istqb-bcs certified tester foundation guide - 4th edition*. BCS Learning & Development Limited.

Academy, C. (2016, May 24). *When Coding Goes Wrong*. Medium.

<https://medium.com/@coderacademy/when-coding-goes-wrong-e46d84c6565f>

The Explosion of the Ariane 5. (2021). Umn.edu. <https://www-users.cse.umn.edu/~arnold/disasters/ariane.html>